Air Quality Index Forecasting for Karachi

A serverless machine learning system that predicts AQI three days ahead

Data Science Internship Project

10Pearls Pakistan

Prepared by

Rizwan Haider

June 2026

Karachi, Pakistan

Built with Python · scikit-learn · XGBoost · SHAP · OpenWeather · Hopsworks · GitHub Actions · Streamlit

Table of Contents

Overview

This project predicts the Air Quality Index (AQI) for Karachi up to three days into the future. The whole system runs on free, serverless tools, so there is nothing to host and no running cost. Live data comes in from a public air quality API, gets turned into model inputs, a model learns from several years of history, and a web dashboard shows the current air quality next to a three-day forecast. People can open the dashboard in a browser and immediately see whether the air is safe and where it is heading.

Where the data comes from

I used the OpenWeather Air Pollution API as the data source. It is free and, importantly, it gives hourly history going back to late November 2020, which is enough to train on. I pulled every hour from 27 November 2020 up to early June 2026, which came to roughly 47,000 hourly readings. Each reading includes eight pollutants: carbon monoxide, nitrogen monoxide, nitrogen dioxide, ozone, sulphur dioxide, fine particulates (PM2.5), coarse particulates (PM10), and ammonia.

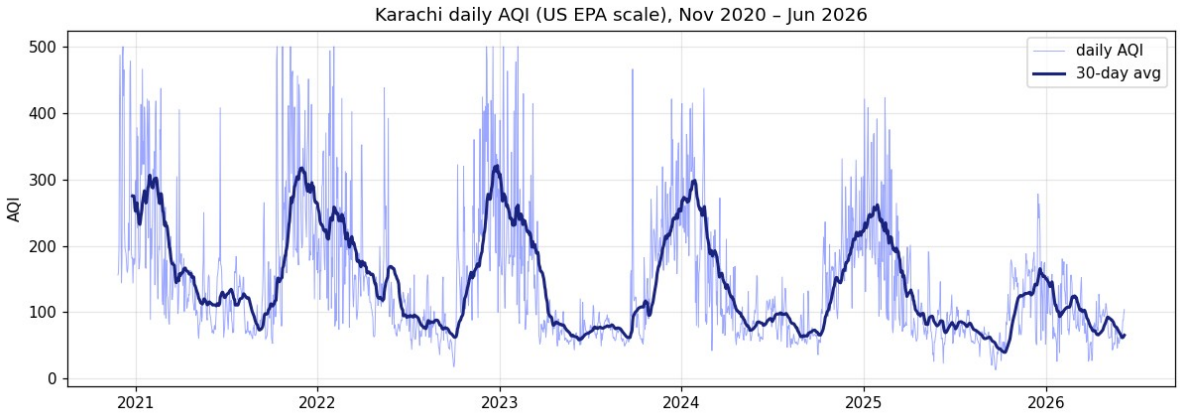
A small number of readings came back as a placeholder value of -9999, which is the API's way of saying “no measurement here”. I replaced those gaps rather than letting them poison the model. After cleaning, I averaged the hourly data into daily values, which gave about 2,000 days to work with. Daily is the right grain for a three-day forecast and it smooths out a lot of hourly noise.

Choosing what to actually predict

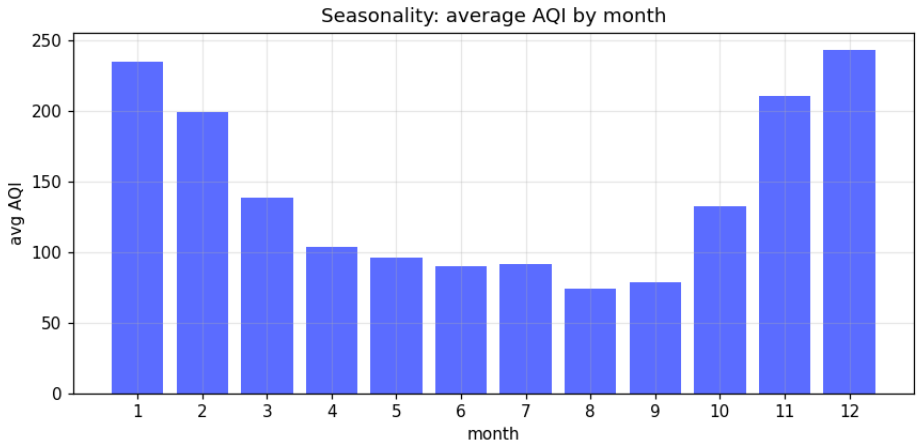
This was one of the more important decisions. OpenWeather reports air quality on a simple 1 to 5 scale (1 is good, 5 is very poor). That scale is too coarse to forecast in a useful way, and the usual accuracy measures barely mean anything on five categories. So instead of predicting their 1 to 5 number, I computed the proper US EPA AQI, the familiar 0 to 500 figure people recognize, directly from the raw pollutant concentrations using the 2024 EPA breakpoints. In Karachi the AQI is driven by particulate matter, so PM2.5 and PM10 set the value. This gave a continuous target that the error metrics describe properly, which made the modelling honest.

What the data shows

Karachi's air is poor on average. The mean daily AQI works out to about 143, which sits in the “Unhealthy for Sensitive Groups” band, with a median of 109. There is a clear seasonal pattern: the winter months from November through January are the worst, while summer is noticeably cleaner, most likely because wind and rain disperse and wash out particulates.



Daily AQI for Karachi from late 2020 to mid 2026, with a 30 day average line.



Average AQI by month, showing the winter peak.

Turning data into features

For each day I built around 60 features for the model. They fall into a few groups: the day's pollutant levels and its computed AQI; calendar features such as month, day of week, day of year and a weekend flag, including sine and cosine versions so the model understands that December and January are neighbors; recent history as lagged values from one, two, three and seven days back, plus seven day rolling averages and variability; and the day to day change in AQI.

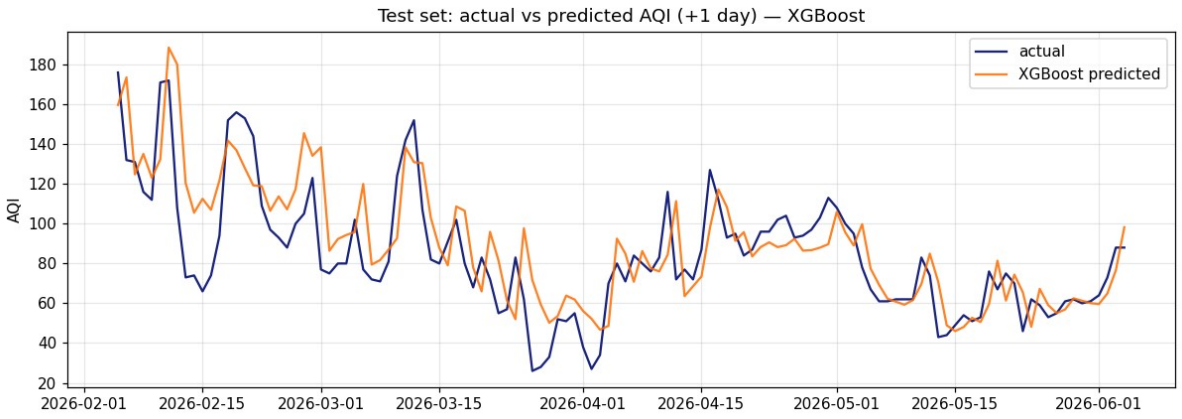
The one rule I was strict about was no leakage. Every feature only uses information that would actually be known on the day a prediction is made. The model never gets to peek at future readings, which is the most common way these forecasts end up looking better in testing than they really are.

Models and results

I trained four models and, crucially, compared them against a plain baseline. The baseline simply guesses that tomorrow's AQI equals today's. That sounds almost too simple, but air quality changes slowly from day to day, so it is genuinely hard to beat. The learned models were Ridge regression, a Random Forest, XGBoost, and a small neural network. I split the data by time, training on the older portion and testing on the most recent 120 days, so the evaluation reflects real forecasting rather than memorising. I scored everything with RMSE, MAE and R squared.

Model	RMSE +1d	RMSE +2d	RMSE +3d	R^2 +1d
Persistence (tomorrow = today)	18.7	27.7	32.1	0.63
Ridge regression	23.9	29.4	30.2	0.39
Random Forest	23.1	31.0	32.0	0.43
XGBoost (best learned model)	20.4	28.0	31.0	0.56
Small neural network	28.1	41.7	51.9	0.16

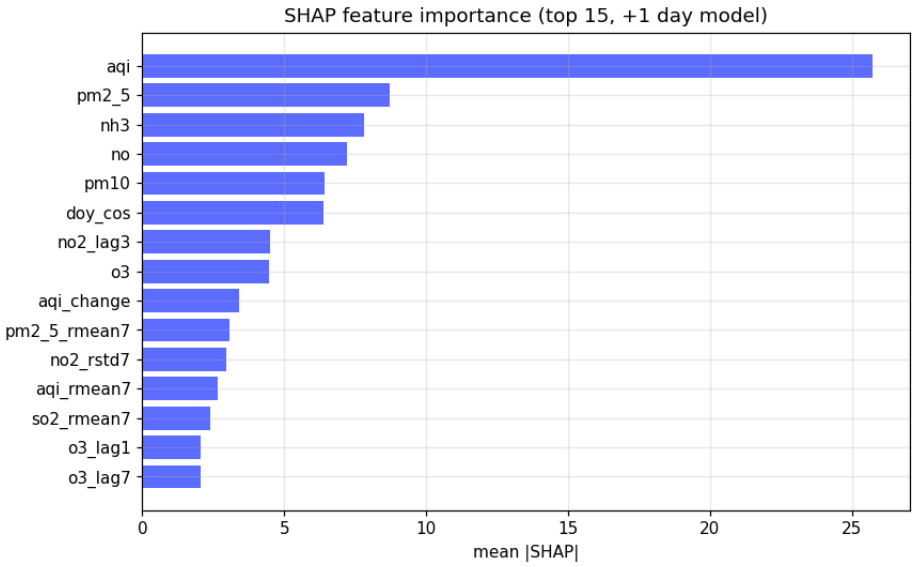
Here is the honest result. XGBoost was the strongest of the learned models, but at one day ahead none of them clearly beat the simple “tomorrow equals today” baseline, and they only became competitive at two and three days out. I even tried having the models predict the change from today rather than the absolute level, and it did not move the needle much. This is a well-known outcome for air quality when you only have pollution data. To beat persistence you need to know what is going to change the air, and the single biggest driver of change is weather, especially wind, which I did not have in this dataset. I treat this as a limitation to build on rather than a failure, and I describe the fix below.



Predicted versus actual AQI on the held out test period (one day ahead).

What the model pays attention to

I used SHAP, a method that explains which inputs push a prediction up or down, to look inside the model. By a wide margin the most important input is the current day's AQI, which fits the slow moving nature of air quality. After that come PM2.5 and PM10, which makes sense because they define the AQI in the first place, along with a couple of the other pollutants. Nothing surprising turned up, which is reassuring: the model is leaning on the things that genuinely matter.



Top features ranked by their average influence on the forecast.

The dashboard

The dashboard is built with Streamlit and deployed for free on Streamlit Community Cloud. When someone opens it, the app pulls the latest data straight from OpenWeather, rebuilds the same features used in training, runs the saved model, and displays the current AQI with its category and colour, a three day forecast, a recent trend chart, a warning banner when unhealthy air is expected, and the top features behind the prediction. It was built to be self contained, so it keeps working even when other parts of the system are unavailable.

Automation and storage

For storage I used Hopsworks, a free feature store and model registry, to hold the engineered features and the trained model. For automation I used GitHub Actions, which runs the pipelines on a schedule with no server to manage: the feature pipeline runs hourly to fetch and store fresh data, and the training pipeline runs daily to retrain and register the best model. The feature pipeline runs and writes data successfully.

An honest note on the automated training step. While I was building this, Hopsworks was migrating its servers to a new version. That migration left one of their background data processing jobs stuck, which in turn blocked the training pipeline from reading the stored history. I deliberately built the dashboard so that it does not depend on that piece, which is why it stays up regardless. Once the platform finishes its migration, that job should clear and the daily training can run end to end.

Challenges I ran into

This project did not go in a straight line, and most of what I learned came from things breaking.

The first wall was just connecting to Hopsworks. I kept getting an "invalid API key" error and could not work out why, until I realised I had copied my account ID from the settings page instead of an actual API key. They look similar but they are not the same thing, and that cost me a fair bit of time. Right after that, the script complained it could not find any project, because I had not created one inside Hopsworks yet. Two small things, but each one stopped me cold until I understood what the error was actually telling me.

Then came a type error I did not expect. The feature store had saved my month, day and weekday columns as one kind of integer, but my pipeline was producing a slightly different kind, and Hopsworks refused to mix them. The numbers were identical, only the internal label was different. Forcing the columns to the matching type fixed it, but it taught me how strict a feature store is about schemas.

The most frustrating problem was not my code at all. After my data uploaded, a background job on Hopsworks that turns the upload into a searchable table got stuck and sat in a "Submitted" state for hours, instead of finishing in a minute or two. Their servers were mid migration to a new version, so jobs were queued and not running. That blocked my training pipeline from reading the history. I decided not to wait on something I could not control, and built the dashboard so it does not depend on that piece at all. It pulls fresh data directly and runs the saved model, so it keeps working regardless.

Deploying the dashboard had its own surprise. The build kept failing because one of my libraries would not install on the newer Python version the hosting platform used. Once I removed that library, which the dashboard did not actually need, the build went through and the app went live.

The modelling result was a challenge of a different kind. My models could barely beat the simple "tomorrow will be like today" guess at one day ahead. That was discouraging at first, but working out why taught me the real lesson. Without weather data, especially wind, the model has no way of knowing what is going to change the air. So the weak spot is not really the model, it is the inputs, and that is the first thing I would add next.

Looking back, the coding was the easy part. Reading errors carefully, working out whether a problem was mine or the platform's, and knowing when to stop fighting something and route around it instead, that is where most of the real learning happened.

Limitations and what I would do next

- Finish the automated training loop once the Hopsworks platform issue described above is resolved.
- The small neural network overfit on a few thousand rows of daily data, so tree based models were the better fit here. With more data, or hourly granularity, a deep model would be worth revisiting.

How to access the project

Source code and pipelines: https://github.com/RizwanHaider44/AQI_Project

Live dashboard: https://aqiproject-9cwbnbcacgjdmndnegj7gf.streamlit.app